

+ 26

CMake

Making Hard Things Easy

BILL HOFFMAN



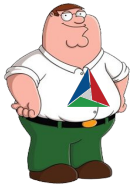
20
26



About Me: Bill Hoffman



- ◆ **1990-1999 GE research in a Computer Vision group.**
 - Moved them from Symbolics Lisp Machines to C++ (CFront) on Solaris and later Linux/Windows
 - Was the build and software library guy (gmake/autotools)
- ◆ **1998-Present Founder and CTO Kitware**
 - Started CMake in 1999
 - Mostly management now (pointy haired boss stuff), finding funding for CMake

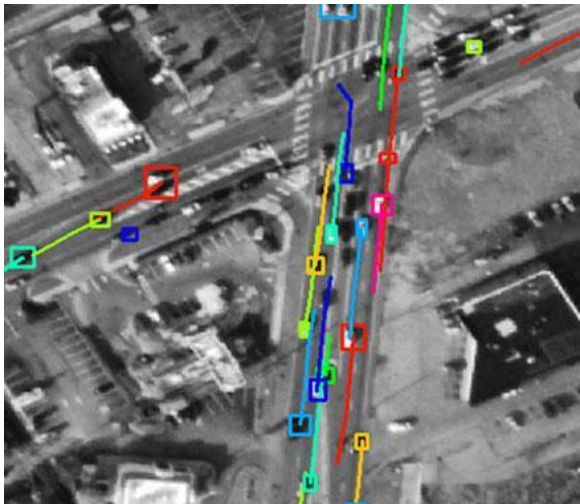


You may know Kitware as the CMake company

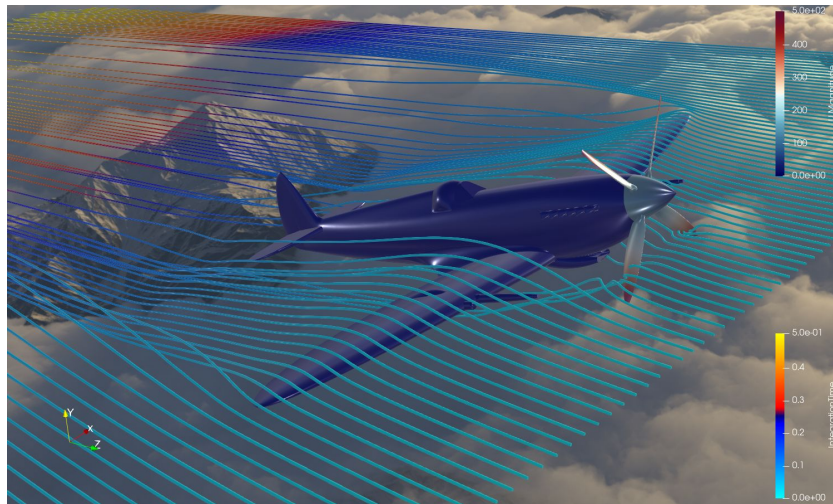


That is understandable—but incomplete.

How Kitware works



Computer vision



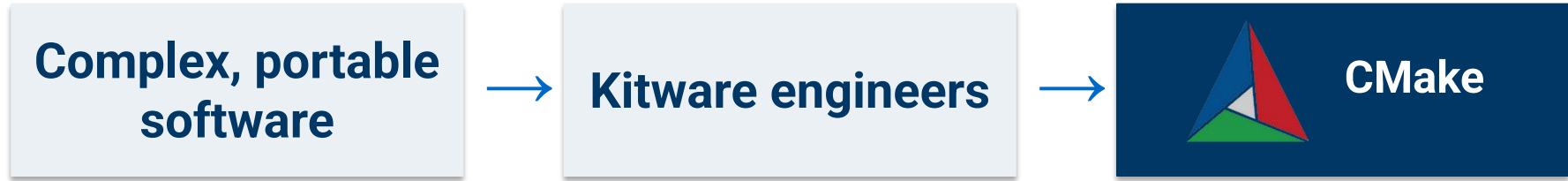
Scientific visualization

Hard technical problem → Expert team → Open-source solution

We provide the expertise and services. The software is open source.

Images: Kitware • Spitfire data: IT4Innovation; rendering: Kitware Europe / ParaView

The build was one of those problems

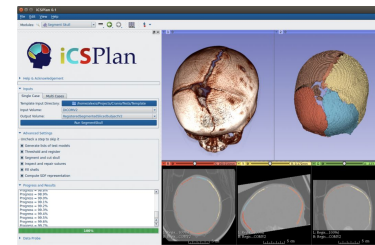
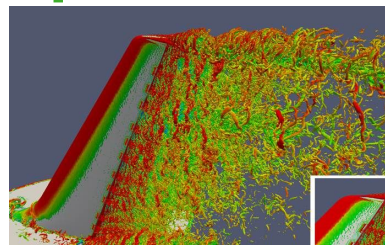
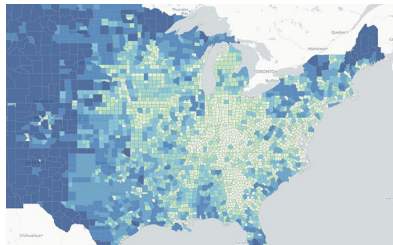
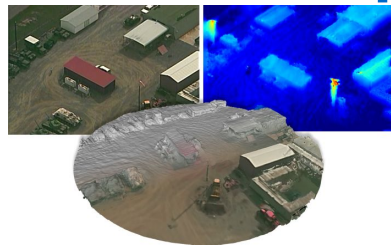


We built CMake so we could build everything else.

Solve a hard problem once. Encode the expertise. Make the next project easier.

Kitware expertise today

Areas of expertise / Built on open source



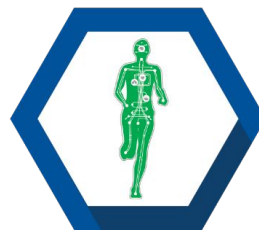
Computer
Vision



Data and
Analytics



Scientific
Computing



Medical
Computing



Software
Solutions



Cyber Security
*New

Kitware Software Solutions Team (CMakeish team)



Bill Hoffman
Chief Technical Officer



Zack Galbreath
Technical Leader



Brad King, Ph.D.
Principal Engineer



Betsy McPhail
Technical Leader



William Allen
R&D Engineer



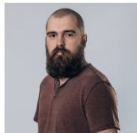
Nicole Cheetham
R&D Engineer



Zhongyuan Chen
R&D Engineer



Martin Duffy
R&D Engineer



Vito Gamberini
R&D Engineer



Andrew Howe
R&D Engineer



Ryan Kottlger
Senior R&D Engineer



Aiden McCormack
R&D Engineer



Tom Oulka
Senior R&D Engineer



John Parent
Senior R&D Engineer



Quinn Powell
R&D Engineer



Taylor Sasser
R&D Engineer



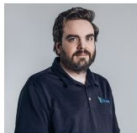
Joseph Snyder
Senior R&D Engineer



Daniel Tierney
R&D Engineer



John Tourtellott
Staff R&D Engineer



Matthew Woehlke
Staff R&D Engineer



Tyler Yankee
R&D Engineer

Small team in 200+ person company

CMake makes hard things easy

CMake turns expert build knowledge into repeatable project behavior.

**Describe the intent
once**

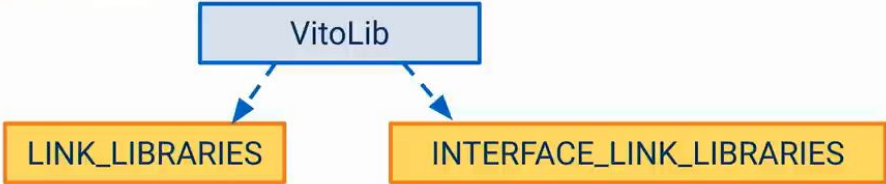
**Automate the difficult
details**

**Keep it working across
tools and platforms**

Recommended: Vito's Post-Modern CMake

CMake 3: I hope you like target_* commands

- PRIVATE target_link_libraries(VitoLib
- INTERFACE [PRIVATE | INTERFACE | PUBLIC]
- PUBLIC <lib>...)



```
graph TD; VitoLib[VitoLib] -.-> LINK_LIBRARIES[LINK_LIBRARIES]; VitoLib -.-> INTERFACE_LINK_LIBRARIES[INTERFACE_LINK_LIBRARIES];
```

kitware

C++ now 2025
Aspen, Colorado

CppNow.org

Video Sponsorship Provided By

Bloomberg

Engineering

think-cell



Post-Modern Cmake
From 3.0 to 4.0

Vito Gamberini

[The four-era framing comes from Vito Gamberini's excellent C++Now 2025 talk. Watch it. 8:22](#)

The eras of CMake

CMake 1

**Primordial Flag
Soup**

2000

CMake 2

World Domination

2004

CMake 3

**I hope you like
target_*
commands**

2014

CMake 4

**Interop and
standardization**

2025
I hope you like JSON



**Following Vito's framing: from generating build systems to defining formats the wider
toolchain can consume.**

From one compiler command to a whole build ecosystem

1 Make it build. Keep it working.

Compilers, platforms, libraries, testing and CI

2 Describe the software.

Targets, usage requirements and IDE integration

3 Connect the ecosystem.

CPS, SBOM and C++ modules

4 Understand what happens.

Debugging and timing the whole workflow

5 Make it faster. Keep it fast.

Measurement, caching and distributed builds

It starts with one file

hello.cpp

One command builds it

hello.cpp

```
gcc  
g++ hello.cpp -o hello
```

A Makefile remembers the command

hello.cpp

GCC

```
g++ hello.cpp -o hello
```

Makefile

```
hello: hello.cpp  
    g++ hello.cpp -o hello
```

Then the project grows

hello.cpp

greeting.cpp

greeting.hpp

Makefile

```
hello: hello.o greeting.o
    g++ hello.o greeting.o -o hello
```

```
hello.o: hello.cpp greeting.hpp
    g++ -c hello.cpp
```

```
greeting.o: greeting.cpp greeting.hpp
    g++ -c greeting.cpp
```

Sources. Headers. Objects. Dependencies.

Same project. One platform. One compiler.

GCC / Linux

```
g++ -O2 -MMD -Iinclude -c  
greeting.cpp  
g++ hello.o greeting.o -o hello
```

Same project. One platform. Two compilers.

GCC / Linux

```
g++ -O2 -MMD -Iinclude -c  
greeting.cpp  
g++ hello.o greeting.o -o hello
```

Vendor compiler

```
CC -fast -Iinclude -c  
greeting.cpp  
CC hello.o greeting.o -o hello
```

Same project. Two platforms.

GCC / Linux

```
g++ -O2 -MMD -Iinclude -c  
greeting.cpp  
g++ hello.o greeting.o -o hello
```

Vendor compiler

```
CC -fast -Iinclude -c  
greeting.cpp  
CC hello.o greeting.o -o hello
```

MSVC / Windows

```
cl /O2 /EHsc /MD /Iinclude /c  
greeting.cpp  
link hello.obj greeting.obj  
/OUT:hello.exe
```

Same project. Three platforms.

GCC / Linux

```
g++ -O2 -MMD -Iinclude -c  
greeting.cpp  
g++ hello.o greeting.o -o hello
```

Vendor compiler

```
CC -fast -Iinclude -c  
greeting.cpp  
CC hello.o greeting.o -o hello
```

MSVC / Windows

```
cl /O2 /EHsc /MD /Iinclude /c  
greeting.cpp  
link hello.obj greeting.obj  
/OUT:hello.exe
```

Apple Clang / macOS

```
clang++ -O2 -Iinclude -c  
greeting.cpp  
clang++ hello.o greeting.o -o  
hello
```

Same project. More details to manage.

GCC / Linux

```
g++ -O2 -MMD -Iinclude -c  
greeting.cpp  
g++ hello.o greeting.o -o hello
```

Vendor compiler

```
CC -fast -Iinclude -c  
greeting.cpp  
CC hello.o greeting.o -o hello
```

MSVC / Windows

```
cl /O2 /EHsc /MD /Iinclude /c  
greeting.cpp  
link hello.obj greeting.obj  
/OUT:hello.exe
```

Apple Clang / macOS

```
clang++ -O2 -Iinclude -c  
greeting.cpp  
clang++ hello.o greeting.o -o  
hello
```

**Compilers, flags, linkers, and runtime rules—
the matrix keeps growing.**

One shared library. Different platform rules.

Linux

```
-fPIC -shared  
libsegment.so.2.1  
SONAME + rpath
```

One shared library. Different platform rules.

Linux

```
-fPIC -shared  
libsegment.so.2.1  
SONAME + rpath
```

macOS

```
-dynamiclib  
libsegment.2.dylib  
install name + @rpath
```

One shared library. Different platform rules.

Linux

```
-fPIC -shared  
libsegment.so.2.1  
SONAME + rpath
```

macOS

```
-dynamiclib  
libsegment.2.dylib  
install name + @rpath
```

Windows

```
/LD  
segment.dll + segment.lib  
exports + DLL placement
```

One shared library. Different platform rules.

Linux

```
-fPIC -shared  
libsegment.so.2.1  
SONAME + rpath
```

macOS

```
-dynamiclib  
libsegment.2.dylib  
install name + @rpath
```

Windows

```
/LD  
segment.dll + segment.lib  
exports + DLL placement
```

**Versioning changes filenames, metadata, compatibility rules,
and runtime lookup differently on every platform.**

Describe the intent once

```
CMakeLists.txt  
add_library(segment SHARED segment.cpp)  
set_target_properties(segment PROPERTIES  
    VERSION 2.1  
    SOVERSION 2)  
target_include_directories(segment PUBLIC include)
```

**The value is not fewer characters. It is shared,
reusable expertise.**

Applications assemble libraries and packages

cURL

TLS • compression •
platform sockets

Applications assemble libraries and packages

cURL

TLS • compression •
platform sockets

VTK

rendering • data models
• many components

Applications assemble libraries and packages

cURL

TLS • compression •
platform sockets

VTK

rendering • data models
• many components

ITK

segmentation •
registration • transitive
libraries

Applications assemble libraries and packages

cURL

TLS • compression •
platform sockets

VTK

rendering • data models
• many components

ITK

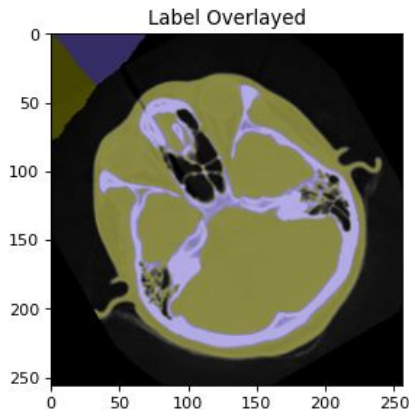
segmentation •
registration • transitive
libraries

What the user had to know

```
include paths + definitions + library order  
debug/release variants + system libraries  
runtime paths + platform-specific details
```

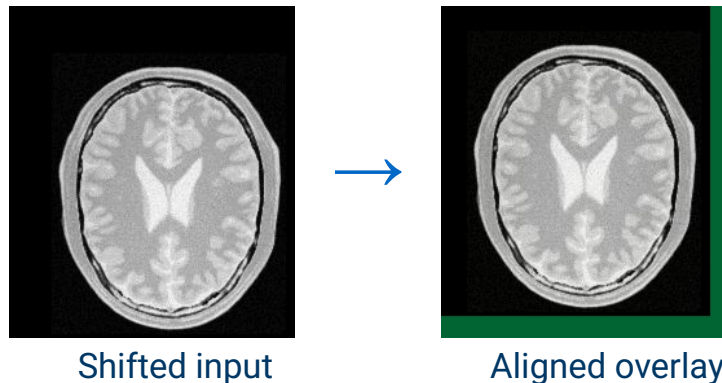
Researchers wanted to solve problems like these

Segmentation



Identify the anatomy.

Registration



Align scans for comparison.

**Their expertise was medical imaging.
Using the software required build expertise, too.**

The user was a researcher, not a linker expert

ITK's purpose

Put state-of-the-art segmentation and registration into the hands of researchers.

The obstacle

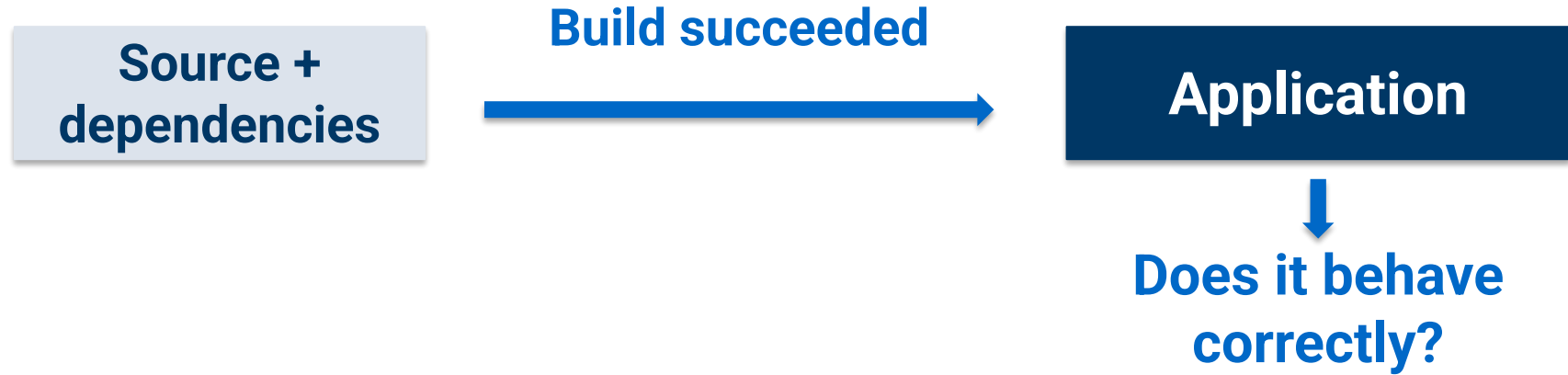
Using a large C++ library required deep compiler and linker knowledge.

CMake moved build expertise from every user into reusable project metadata.

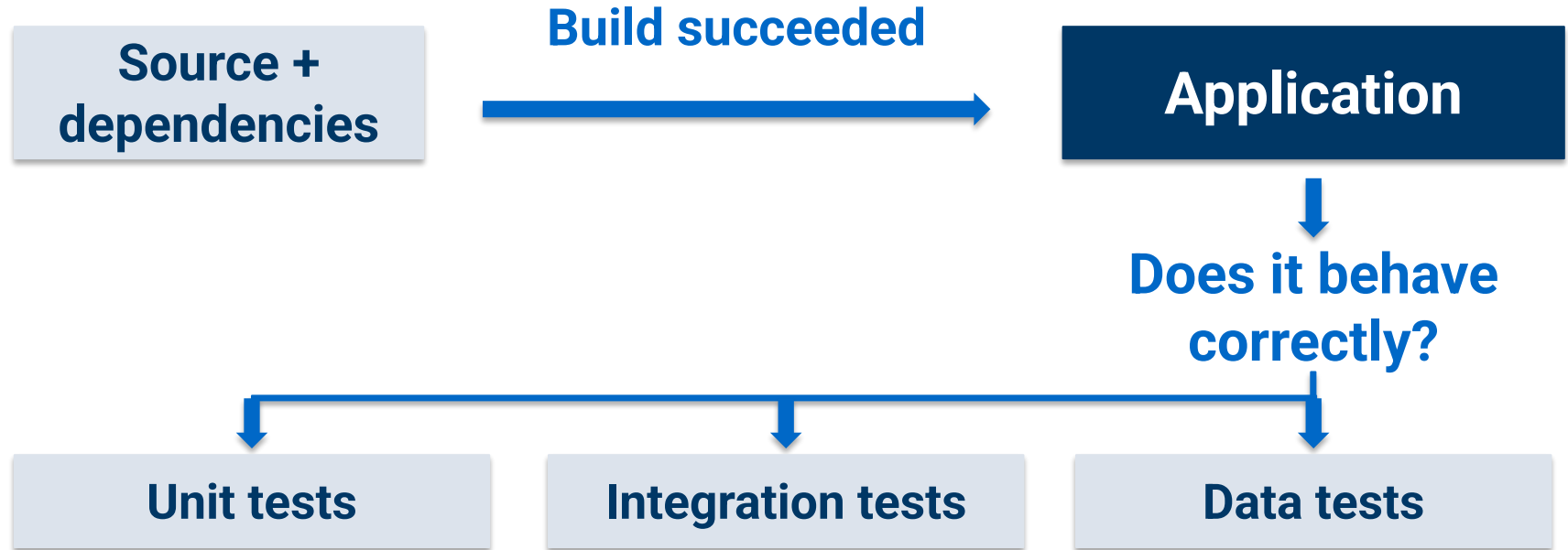
It builds. Does it work?



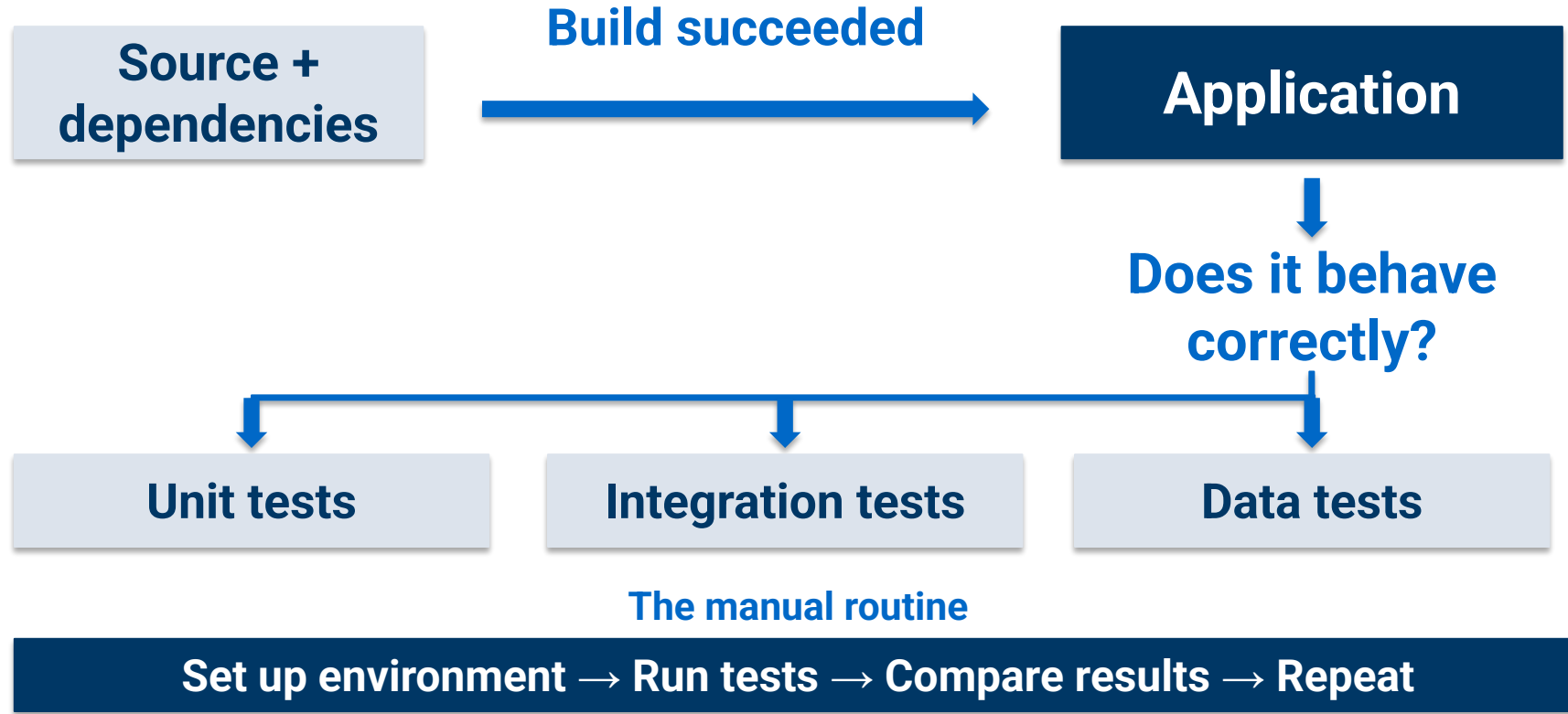
It builds. Does it work?



It builds. Does it work?



It builds. Does it work?



The manual routine

Set up environment → Run tests → Compare results → Repeat

Across platforms, after every change

Describe the expected behavior once

CMakeLists.txt

```
enable_testing()  
add_test(NAME segment_test  
         COMMAND segment_test  
         input.nrrd)
```

Every build tree

```
ctest --output-on-failure
```

**Versioned with the
project**

**Repeatable by developers
and CI**

**Consistent across
platforms**

Build, test, report, repeat

Linux

macOS

Windows



Scan Build											1 build
Site	Build Name	Update	Configure		Build						
		Revision	Error	Warn	Error	Warn	Start Time ▼				
elysium-linux.kitware	Linux-clang-scanbuild	8c3071	0	0	0	0	10 hours ago				
Nightly Expected											96 of 108 builds
Site	Build Name	Update	Configure		Build		Test			Start Time ▼	
		Revision	Error	Warn	Error	Warn	Not Run	Fail ▼	Pass		
trinsic.kitware	vs14-64-ninja	8c3071	0	0	0	0	0	2 ⁺² ₋₂	479 ₋₂	13 hours ago	
trinsic.kitware	mingw64-make	8c3071	0	0	0	0	0	1 ⁺¹ ₋₁	449 ₋₁	6 hours ago	
hythloth.kitware	Linux-ninja-gcov	8c3071	0	0	0	0	0	1	519	7 hours ago	
trinsic.kitware	vs14-64-ide-v90	8c3071	0	0	0	0	0	1 ⁺¹ ₋₁	387 ₋₁	8 hours ago	
vesper.kitware	watcom	8c3071	0	0	0	0	0	1 ⁺¹ ₋₁	368 ₋₁	8 hours ago	
trinsic.kitware	vs9-64-ide	8c3071	0	0	0	0	0	1 ⁺¹ ₋₁	381 ₋₁	9 hours ago	
vesper.kitware	bcc55	8c3071	0	0	0	0	0	1 ⁺¹ ₋₁	370 ₋₁	9 hours ago	
dash3win7.kitware	vs14-64	8c3071	0	0	0	0	0	1 ⁺¹ ₋₁	460 ⁺¹ ₋₁	9 hours ago	
dashsun1.kitware.com	Solaris-10-8.11_Oracle-12.3	8c3071	0	0	0	0	0	0	415	1 hour ago	

Portability means continuously proving that it works everywhere.

Modern CMake describes your software

What do I need to build?

image library

What does it need?

Sources
Include directories
Language requirements
Dependencies

viewer app

uses

image library

Put the knowledge with the target that owns it.

A target carries its requirements

CMakeLists.txt

```
add_library(image image.cpp)
target_include_directories(image PUBLIC include)
target_compile_features(image PUBLIC cxx_std_20)
target_compile_definitions(image PRIVATE IMAGE_INTERNAL)
```



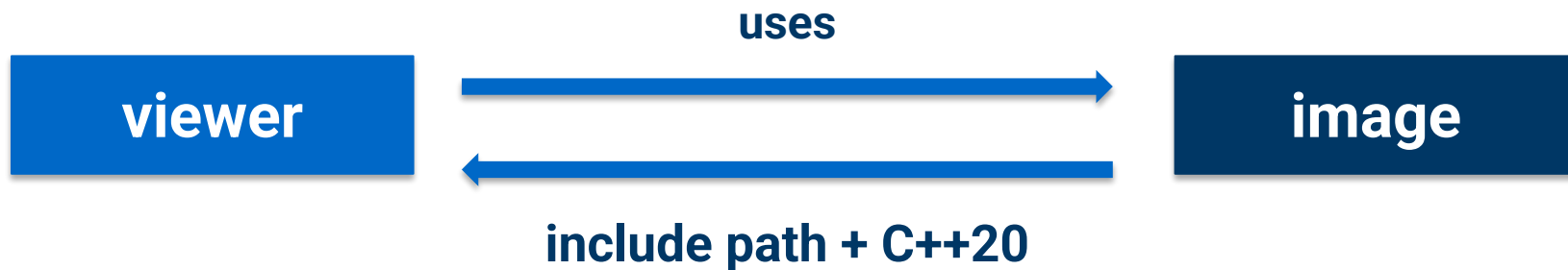
PUBLIC: this target and its consumers

PRIVATE: this target only

Declare the dependency. Carry the requirements.

CMakeLists.txt

```
add_executable(viewer viewer.cpp)
target_link_libraries(viewer PRIVATE image)
```



Change the requirement at its source.
Connected consumers get the updated requirement.

No copied include paths. No duplicated compiler flags.

IDE integration needs CMake's model

CMake 3.14

Extra generators

One IDE-specific output

CodeBlocks - Ninja
Eclipse CDT4 - Ninja
Kate - Unix Makefiles

deprecated in CMake 3.27

File API



codemodel | cache | toolchains |
configure log

One supported, versioned integration
surface

The File API is a versioned conversation

1 IDE writes a query

query

```
.cmake/api/v1/query/  
client-ide/query.json
```

2 CMake configures and generates

**CMake reads the
requested object kinds**

3 IDE reads the reply index

reply

```
.cmake/api/v1/reply/  
index-*.json
```

codemodel-v2

cache-v2

toolchains-v1

configureLog-v1

**Tools consume CMake's semantic model without parsing
generated build files.**

CPS and SBOM

CPS Common Package Specification

SBOM Software Bill of Materials

The software model crosses project boundaries



CppCon 2023: a call to action for CPS



**“Somebody needs
to do something
about this.”**

Bret Brown + Bill Hoffman • CppCon 2023

Libraries: A First Step Toward
Standard C++ Dependency Management

CppCon 2023: a call to action for CPS



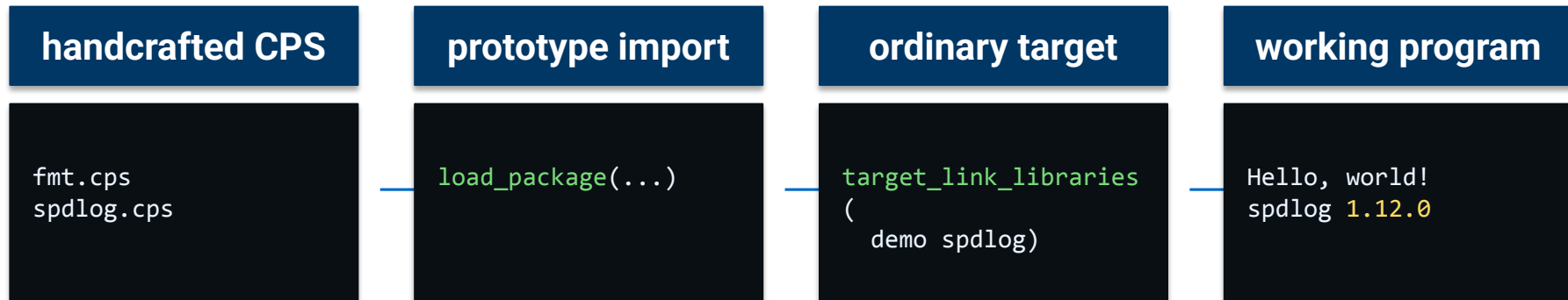
**“Somebody needs
to do something
about this.”**

**“Hey, I’m
somebody. Let’s do
something.”**

Bret Brown + Bill Hoffman • CppCon 2023

Libraries: A First Step Toward
Standard C++ Dependency Management

The 2023 demo proved the handoff



A JSON file carried the producer's usage requirements to the consumer.

The prototype called `load_package()`. The promised destination was `find_package()`.

CPS describes how to consume a package

Consumer

```
find_package(ExamplePackage REQUIRED
  COMPONENTS Core Rendering)
target_link_libraries(app PRIVATE
  ExamplePackage::Rendering)
```

- libraries and components
- versions and compatibility
- public dependencies
- compile and link requirements

Here are my libraries. Here is how to use them correctly.

[Specification: github.com/cps-org/cps](https://github.com/cps-org/cps)

[CMake docs: find_package\(\)](#)

[install\(PACKAGE_INFO\)](#)

CPS turns package code into data

CMake package logic

```
find_dependency(zlib-ng)
conditions
platform logic
```

CPS

```
"requires": {
  "zlib-ng": {}
}
```

Neutral, declarative, machine-readable package metadata

Change the package format—not the consumer

Before — produce CMake code

```
install(TARGETS zwrap
        EXPORT zwrapTargets)

install(EXPORT zwrapTargets
        FILE zwrapConfig.cmake
        NAMESPACE zwrap::
        DESTINATION lib/cmake/zwrap)
```



Now — CMake 4.3 produces CPS data

```
install(TARGETS zwrap
        EXPORT zwrap)

install(PACKAGE_INFO zwrap
        EXPORT zwrap)
# installs zwrap.cps
```

Consumer — unchanged

CMakeLists.txt

```
find_package(zwrap REQUIRED)
target_link_libraries(app PRIVATE zwrap::zwrap)
```

CPS is ready for early adopters. Start publishing it.

SBOM records what was delivered

application 1.4

cURL 8.8

OpenSSL 3.3

zlib-ng 2.2

components | versions | relationships | suppliers | licenses | hashes

A machine-readable inventory and provenance record for the delivered product.

An SBOM answers two operational questions

Security response

A vulnerability is found
in component Y, version X

Do any of our delivered
products contain it?

License compliance

The product includes many
third-party components

Which licenses—and
obligations—are we shipping?
Especially important for reciprocal/copyleft
licenses such as GPL.

Same inventory. Two different questions.

A vulnerability is reported in libY 2.4

Security advisory

**libY 2.4 is
affected**

**Search delivered
product SBOMs**

**Product A
contains libY 2.4
INVESTIGATE**

**Product B
contains libY 2.6
NO VERSION MATCH**

The SBOM identifies delivered products that contain the affected component and version.

Two ways to generate an SBOM

Experimental in CMake

Use existing export sets

Core → SBOM

Apps → SBOM

Quick starting point

Define the package

Core + Apps



Product SBOM

Control scope and metadata

Both routes require experimental SBOM support to be enabled.

Start with the exports you already have

Prerequisite / before project()

```
set(CMAKE_EXPERIMENTAL_GENERATE_SBOM  
    "<UUID required by your CMake version>")
```

[Use the UUID from your version's experimental-feature documentation](#)

Existing install export set

```
install(TARGETS core codec EXPORT CoreTargets)
```

Configure / then build and install as usual

```
cmake -S . -B build -DCMAKE_INSTALL_SBOM_FORMATS=SPDX
```

One SBOM per export set. Existing export sets are required.

Export-set boundaries may not match your delivered product.

Define one SBOM for the package

With experimental SBOM support enabled

```
CMakeLists.txt  
install(TARGETS core codec EXPORT CoreTargets)  
install(TARGETS viewer EXPORT AppTargets)  
  
install(SBOM ImagingSuite  
  EXPORTS CoreTargets AppTargets  
  VERSION 2.0  
  DESCRIPTION "Imaging tools"  
)
```

core + codec + viewer → ImagingSuite SBOM

Choose the export sets, package name, version and metadata.

What goes into the document?

SPDX JSON-LD / component / selected fields

```
{ "type": "software_Package",  
  "spdxId": "urn:viewer#Package",  
  "name": "viewer",  
  "software_packageVersion": "2.0" }
```

A component

**Identifies viewer
and its version**

SPDX JSON-LD / relationship / selected fields

```
{ "type": "Relationship",  
  "relationshipType": "dependsOn",  
  "from": "urn:viewer#Package",  
  "to": [ "urn:codec#Package",  
          "urn:core#Package" ] }
```

Its dependencies

**viewer depends on
codec and core**

Generated by CMake for ImagingSuite. Other fields omitted.

Start quickly. Shape it for your product.

Try it

Enable automatic SBOMs

Inspect the output



Prepare it for delivery

Choose the export sets
Supply package metadata
Review component coverage

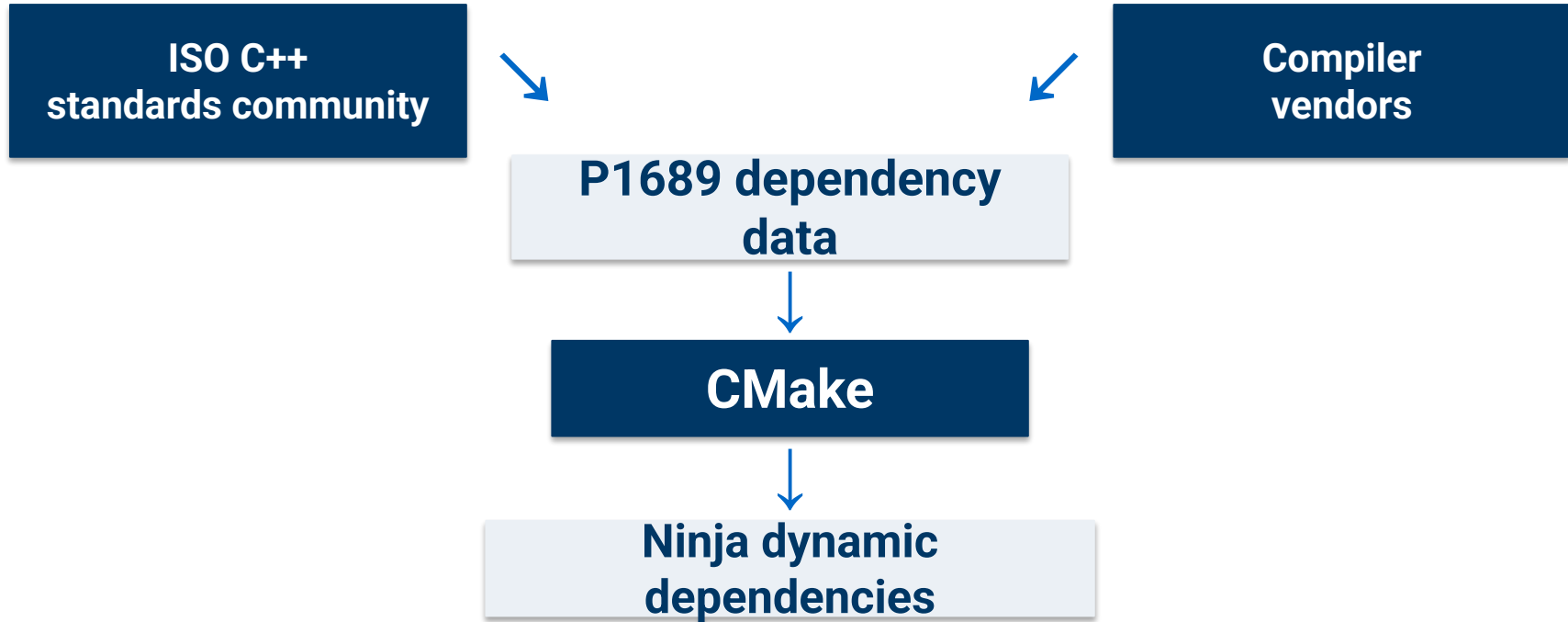
Generate it with every installation.

Experimental: try it, inspect it, and share feedback.

C++ modules

It takes a village.

C++ modules could not be solved by CMake alone



No single tool owned enough of the problem to solve it alone.

2005: CMake's first module dependency scanner

Fortran—not C++

```
geometry.f90
module geometry
...
end module geometry
```

→
geometry.mod
→

```
solver.f90
module solver
use geometry
...
```

CMake contains its own Fortran dependency parser.
It reads MODULE and USE to discover the build order.

CMake's first foray into language modules.

Before CMake understood Fortran modules...

An old Fortran module workflow

```
$ make  
fatal error: cannot open geometry.mod
```

make #1

Before CMake understood Fortran modules...

An old Fortran module workflow

```
$ make  
fatal error: cannot open geometry.mod  
  
$ make  
fatal error: cannot open solver.mod
```

make #1

make #2

Before CMake understood Fortran modules...

An old Fortran module workflow

```
$ make  
fatal error: cannot open geometry.mod
```

```
$ make  
fatal error: cannot open solver.mod
```

```
$ make  
[100%] built simulation
```

make #1

make #2

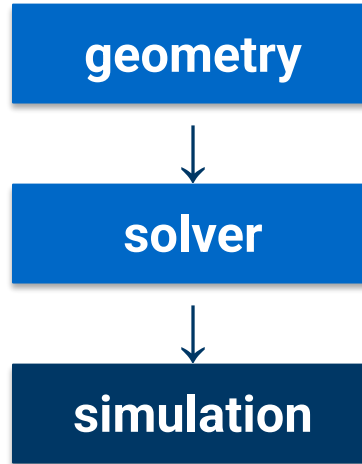
make #3

Keep running make until the build converges.

CMake turned Fortran modules into a build graph

dependency scanner •
2005

```
CMakeLists.txt  
add_executable(simulation  
  geometry.f90  
  solver.f90  
  simulation.f90  
)
```



CMake discovers the module dependencies and schedules each compilation in the correct order.

2016: the Fortran model reaches Ninja dyndep

Configure

```
$ cmake -S . -B build -G Ninja
-- Configuring done
-- Generating done
```

Ninja output (paths shortened)

```
$ ninja -C build
[1/8] Building Fortran preprocessed ...geometry...
[2/8] Building Fortran preprocessed ...simulation...
[3/8] Building Fortran preprocessed ...solver...
[4/8] Generating Fortran dyndep file .../Fortran.dd
[5/8] Building Fortran object ...geometry.f90.o
[6/8] Building Fortran object ...solver.f90.o
[7/8] Building Fortran object ...simulation.f90.o
[8/8] Linking Fortran executable simulation
```

**CMake Fortran
parser**

scans USE and MODULE
statements



.ddi records

capture discovered
dependencies



Fortran.dd

updates Ninja's graph with
dyndep



Ninja

builds geometry → solver →
simulation

**The build graph is corrected before the
dependent compilations run.**

Modules make build order part of compilation

B.cpp

```
export module B;  
export void b() { }
```

A.cpp

```
export module A;  
import B;  
export void a() { b(); }
```

A cannot compile until B's module interface exists.

Build A before B

```
> cl -std:c++20 -interface -c A.cpp  
A.cpp(2): error C2230: could not find module 'B'  
A.cpp(3): error C3861: 'b': identifier not found
```

For C++, we needed the compiler

FORTTRAN

CMake's own dependency
parser
reads MODULE and USE



CMake discovers
module dependencies

C++

Maintaining a C++ parser
inside
CMake was not practical



Work with compiler vendors
to expose dependency
scanning

P1689: a common format
for the dependency results

Same scheduling problem. Different discovery mechanism.

Discover the dependencies. Then compile.

1 SCAN

Compiler examines
the sources

B.cpp provides B

A.cpp imports B

P1689
dependency
data

2 CONNECT

CMake combines
the results

A needs B

Ninja
dyndep

3 BUILD

Ninja schedules
compilation

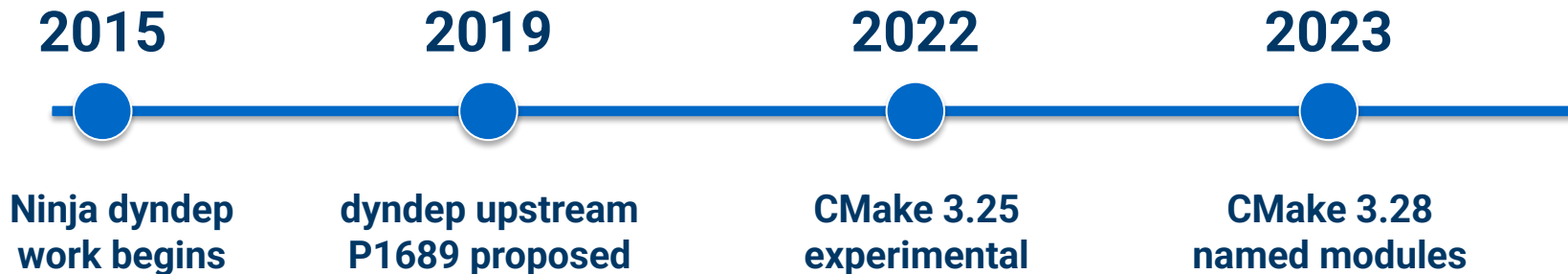
Compile B

Compile A

The dependency is discovered before A compiles.

Years of collaboration made this practical

Compiler vendors + C++ standards work + CMake + Ninja



**Your project declares the module files.
The toolchain handles the coordination.**

Are we modules yet?

Are We Modules Yet?

Home Projects Tools Docs Examples Bluesky GitHub

C++20 modules for a brighter future – and your help!

Faster compilation • Better encapsulation • No macro leakage • No include order issues



228 of 2,613 projects complete • Estimated completion: 23 January 2085

Stars updated 8 Sep 2026 • vcpkg data 7 Sep 2026 (commit) • 425 C libraries excluded • Rebuilt 8 Sep 2026

This website tracks the most popular C++ projects, compilers, and build systems. Projects are ranked by GitHub stars by default, with vcpkg port updates used when GitHub data is unavailable. You can also switch the ranking to prefer vcpkg data.

We also have an example section to track good projects using modules natively and a document section tracks good articles about modules too.

Feel free to contribute to these lists.

Browse all projects



© 2026 Are We Modules Yet? • Powered by Hugo & PaperMod • Jekyll

✓	poco	import Poco;	★ 9,486	1.14.1	2026-02-10	
✓	seastar	import seastar;	★ 9,359		2023-03-24	
✓	tbb	import tbb;	★ 6,740	2023.1.0	2026-05-13	
✓	magic-enum	import magic_enum;	★ 6,187	0.9.8	2024-05-09	
✓	MaaFramework		★ 4,835		2024-03-19	
✓	✓ infinity		★ 4,784		2025-05-15	
✓	clili	import clili;	★ 4,439	2.7.2	2026-02-10	
✓	mengraph	import mengraph.*;	★ 4,406		2025-10-01	
✓	ctre	import ctre;	★ 3,844	3.11.0	2024-05-17	
✓	vulkan-hpp	import vulkan;	★ 3,788	deprecated	2023-06-23	
✓	argparse	import argparse;	★ 3,542	3.2	2023-10-15	
✓	✓ GTAIV_EFLC.FusionFix		★ 3,230		2023-10-28	
✓	bshoshany-thread-pool	import BS_thread_pool;	★ 3,050	5.1.0	2024-12-19	
✓	quill	import quill;	★ 3,025	13.0.0	2026-06-13	
✓	skfft	import Strata.*;	★ 2,980		2025-10-05	
✓	hpx	import HPX.*;	★ 2,890	1.11.0	2024-09-06	
✓	openal-soft	import openal.*;	★ 2,742	1.25.1	2025-09-02	
✓	sqlite-orm	import sqlite_orm.*;	★ 2,683	1.9.1	2025-02-06	
✓	kokkos	import kokkos.*;	★ 2,673		2025-10-01	
✓	stdexec	import stdexec;	★ 2,429	Unknown	2025-07-27	
✓	async-simple	import async_simple;	★ 2,197	1.4	2023-06-19	
✓	tomplusplus	import tomplusplus;	★ 2,142	3.4.0	2025-04-01	

Hey, I know Kokkos developers.

12–21% faster builds

Release **12%**

RelWithDebInfo **21%**

Debug **17%**

Kokkos 5.1.0 • CMake 4.3.2 • Ninja 1.11.1 • Clang 19.1.7

The real module

```
import kokkos.core;  
#include <Kokkos_Macros.hpp> // sigh...
```

Toolchain reality

Clang 22 and GCC 15 did not work for this test.

The successful run used the CI containerized toolchain.

Real gains—and still-fragile toolchains.

Kokkos only had to describe its modules

Kokkos — core/src/CMakeLists.txt

```
if(Kokkos_ENABLE_EXPERIMENTAL_CXX20_MODULES)
  set(KOKKOS_CORE_MODULE_FILES
      Kokkos_Core.cppm Kokkos_Core_Impl.cppm)
endif()

kokkos_add_library(kokkoscore
  MODULE_INTERFACE ${KOKKOS_CORE_MODULE_FILES}
  ...)
```

CMake 3.28

**Kokkos names the
module interface files.**

**CMake and Ninja handle
scanning, dependency
discovery, and build
order.**

-DKokkos_ENABLE_EXPERIMENTAL_CXX20_MODULES=ON

Using import std with CMake

CMakeLists.txt

```
add_executable(hello main.cpp)
target_compile_features(hello PRIVATE
cxx_std_23)
set_property(TARGET hello PROPERTY
CXX_MODULE_STD ON)
```

main.cpp

```
import std;

int main() {
    std::println("Hello, CppCon!");
}
```

configure

```
cmake -S . -B build -G Ninja \
  -DCMAKE_EXPERIMENTAL_CXX_IMPORT_STD=<gate>
```

The mechanism worked—but users had to know how to unlock it.

Using import std with CMake

CMakeLists.txt

```
add_executable(hello main.cpp)
target_compile_features(hello PRIVATE
cxx_std_23)
```

main.cpp

```
import std;

int main() {
    std::println("Hello, CppCon!");
}
```

configure

```
cmake -S . -B build -G Ninja
```

Write import std; CMake handles the rest.

Want to understand what happens underneath?

The familiar build

translation unit → object file →
link

The modules build

scan → collate → BMI →
schedule

Building C++20 Modules: The Rest of the Story

Vito Gamberini

Vito covered the rest of this story earlier at CppCon.
Watch his talk online.

CMake 4.5: C++ module support

Compiler	Named modules	import std
MSVC	14.34+	14.36+
Clang	16+	18.1.2+
GCC	14+	15+*
Apple Clang	—	—

The conspicuous gap: Apple Clang from Xcode

* GCC 16+ on macOS

CMake became complex, too.

Managing that complexity required better tools.

Why does it do that?

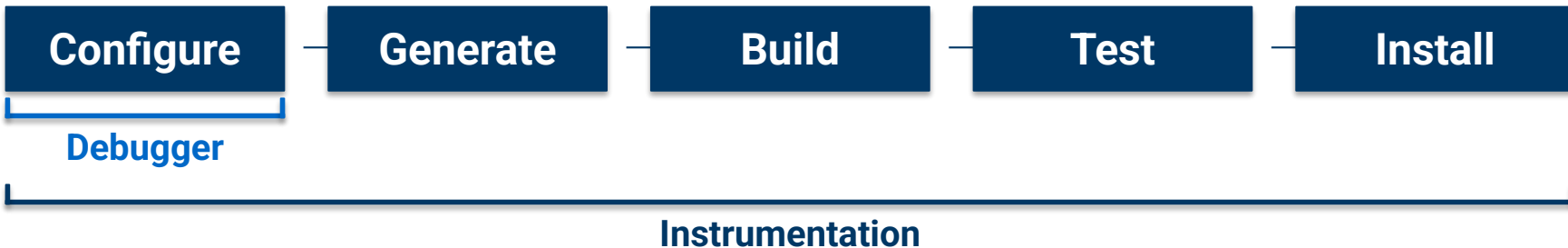
A real CMake debugger

Breakpoints. Step. Inspect.

Where does the time go?

Build instrumentation

Measure the work CMake orchestrates.



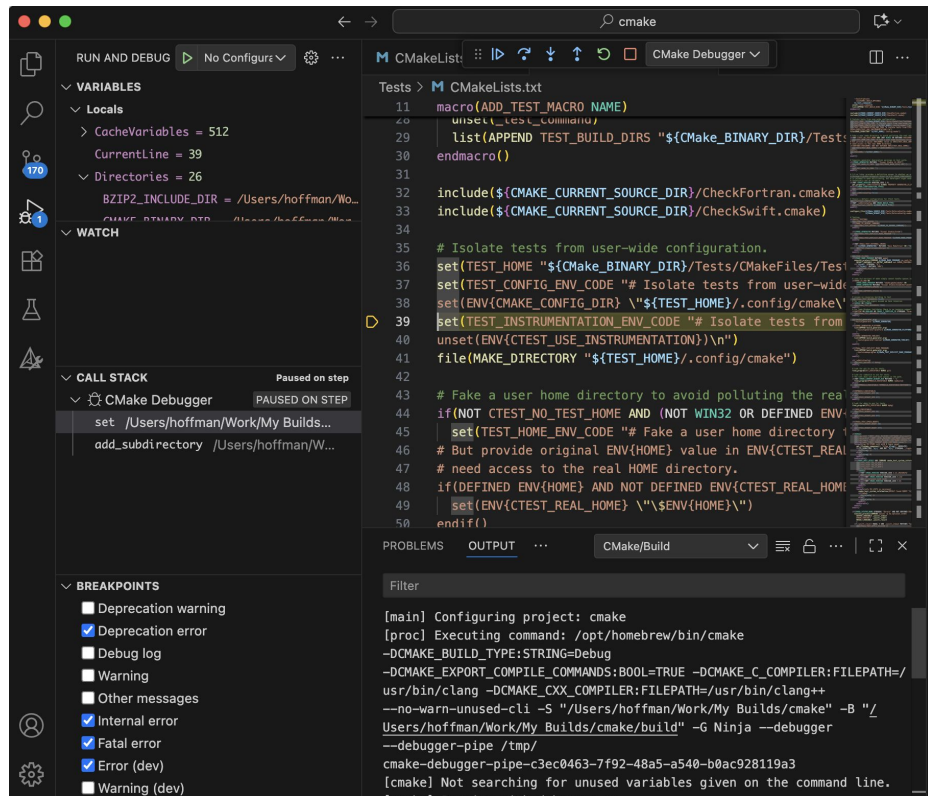
A real debugger for CMake

Stop guessing. Inspect CMake code.

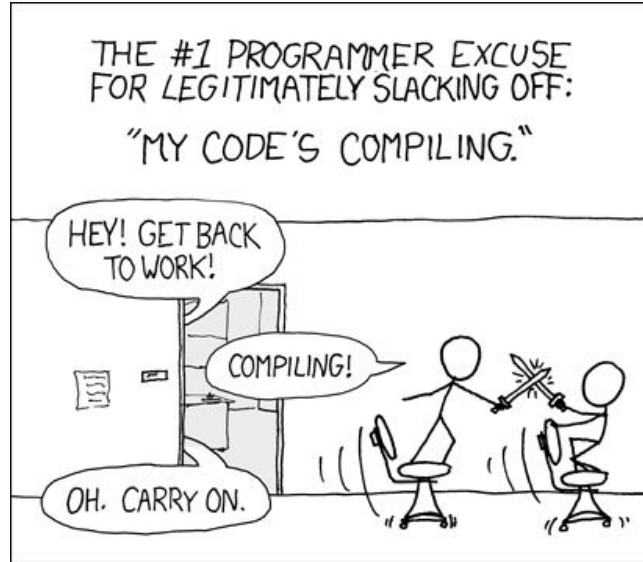
- Set breakpoints
- Step through CMake code
- Inspect variables
- Follow the call stack

Debug Adapter Protocol (DAP)

Shown in VS Code



Where does the time go?



Configure

Generate

Build

Test

Install

What are we waiting for?

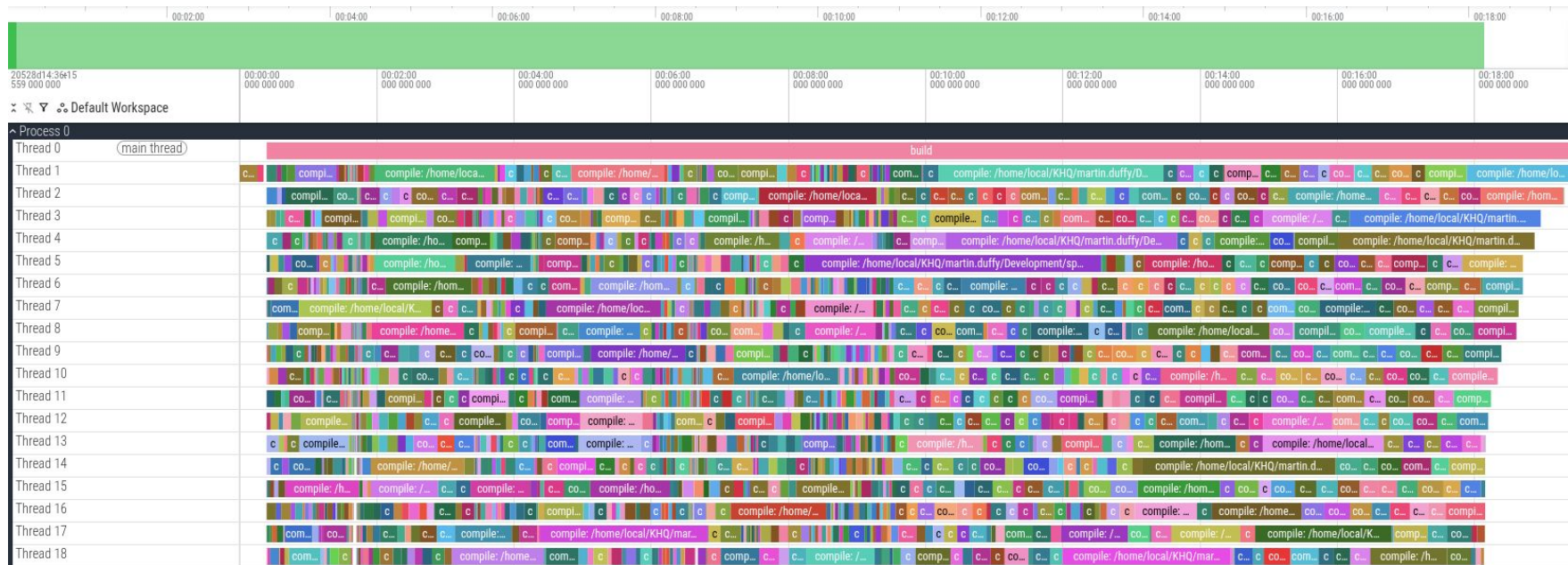
Without instrumentation

Terminal

```
$ cmake -S . -B build -G Ninja
-- Configuring done (30.2s)
-- Generating done (5.2s)

$ cmake --build build
[3041/3041] Linking CXX static library project/libproject.a
```

Finished. But what made us wait?



Look inside the elapsed time.

Trilinos build / instrumentation-generated Google Trace Event visualization

Follow the time through the whole workflow

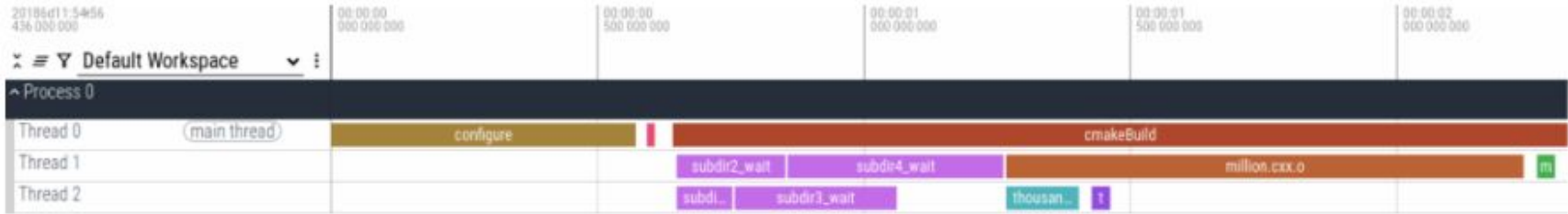


Timing + commands + targets

Optional CPU-load and memory measurements

Which stage is slow? What work dominates it?

Find what is holding compilation back



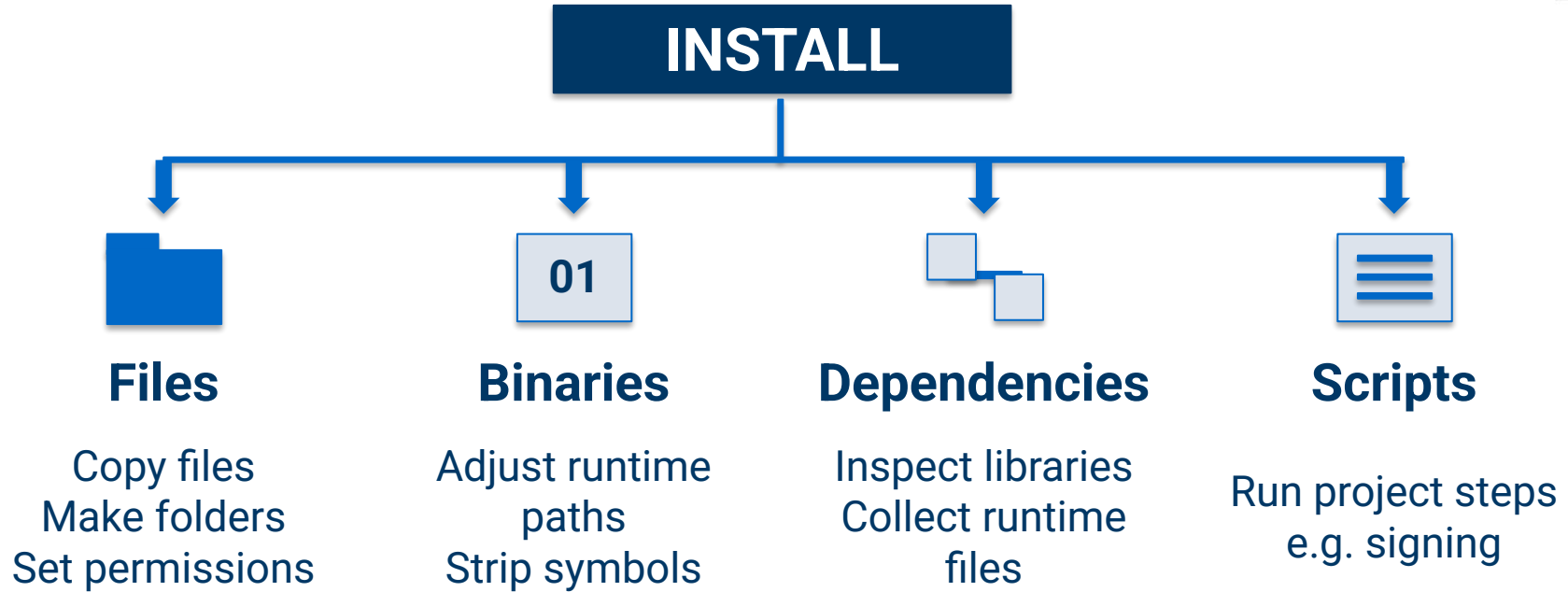
Compilation waits on dependent targets

One possible remedy

```
set_property(TARGET mylib PROPERTY OPTIMIZE_DEPENDENCIES ON)
```

Removes unnecessary dependencies of static and object libraries.

Why are we still waiting after the build?



Independent directories can do this work in parallel.

Depending on the project and platform

The build is done. The waiting may not be.

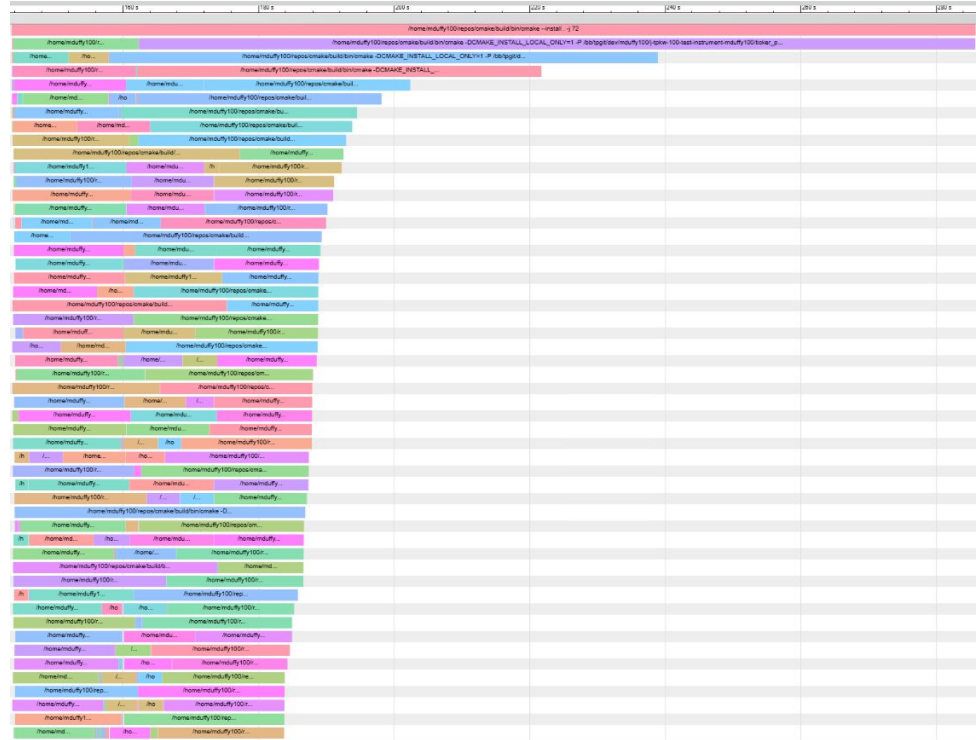
Install has a long tail

Most work finishes.

A few directories take longer.

Enable parallel install.
Then measure again.

Tests have long tails, too.
Find the slowest tests.



Start collecting measurements

CMakeLists.txt / CMake 4.3+

```
cmake_instrumentation(  
  API_VERSION 1  
  DATA_VERSION 1  
  HOOKS postGenerate  
        postCMakeBuild  
        postCTest  
        postCMakeInstall  
  OPTIONS trace  
          dynamicSystemInformation  
)
```

Run your normal workflow

Commands

```
cmake -S . -B build -G Ninja  
cmake --build build  
ctest --test-dir build  
cmake --install build
```

Each hook collects new data.
Open the trace in a viewer.

The same data can serve a whole team



A hook passes the index path to your callback.



Local analysis

Read JSON. Diagnose this build.



Central history

Upload JSON. Compare builds.

CMake supplies the data and callback hooks; your tooling supplies upload, storage and reporting.

Where is your team spending build time?



Hundreds of developers. Shared evidence.

Where is your team spending build time?



Hundreds of developers. Shared evidence.

**Which stages
dominate?**

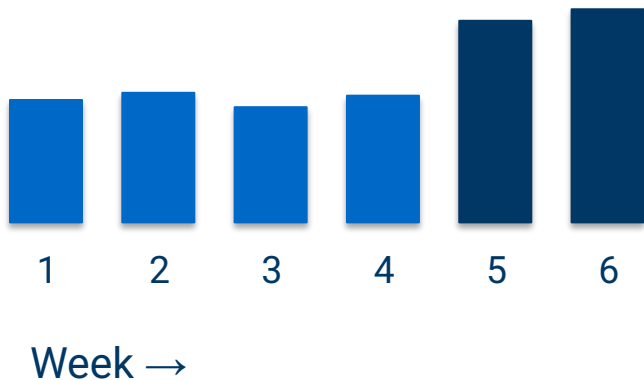
**Which
configurations hurt?**

**When did it get
slower?**

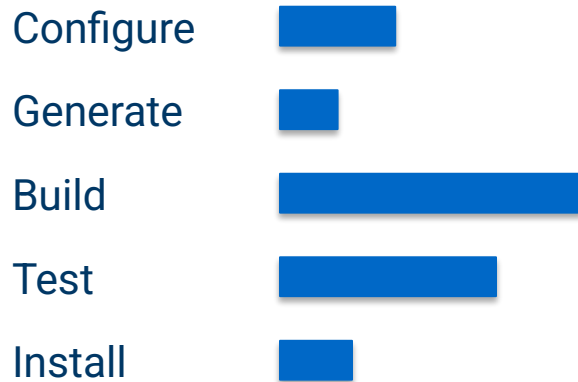
Keep it fast. Catch regressions.

Illustrative reporting / incremental Debug builds, same toolchain

Typical elapsed time



Time by stage



Compare like builds. Find the change. Verify the fix.

Measure once. Put the data to work.

Fix it now

One build

Find the bottleneck

Keep it fast

Build history

Detect regressions

Improve the team

Aggregate patterns

Prioritize investment

Choose acceleration that addresses the bottleneck.

Build faster with CMake

Reuse work

Compiler caches

ccache / sccache

Distribute work

FASTBuild

recc + BuildBox

CMake RE

FASTBuild and CMake RE also provide caching.

Two ways to shorten a build

Reuse work

Same inputs



Cached result

ccache / sccache

Share the work

Independent compilations



Other machines compile

distcc

CMake works with both kinds of tools.

Compiler caching fits your CMake build

Compiler launcher / existing Ninja build

```
cmake -S . -B build -G Ninja \  
-DCMAKE_CXX_COMPILER_LAUNCHER=sccache
```

Kitware Linux CI



Reuse compilation results across CI jobs.

[Kitware Linux CI / shared cache configured separately from the compiler launcher](#)

FASTBuild: start with a different generator

CMake 4.2+

```
cmake -S . -B build -G FASTBuild
```

CMake → fbuild.bff → FASTBuild

Distribute / workers configured

```
cmake --build build -- -dist
```

Cache / cache location configured

```
cmake --build build -- -cache
```

Configure worker discovery, toolchain files and the cache location.

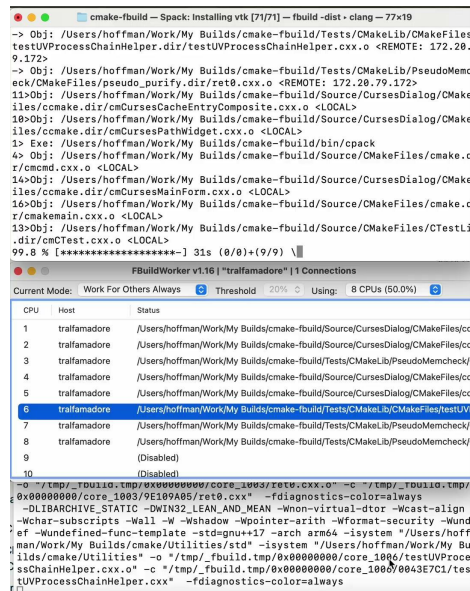
FASTBuild: distribute the work

Workers configured

```
cmake --build build -- -dist
```

Local + remote compilation

Another machine takes
compilation jobs.



The screenshot shows a terminal window with FASTBuild output and a separate window titled "FBUILDWorker v1.16 | 'traffamadore' | 1 Connections".

Terminal Output:

```
cmake-fbuild -- Spack: Installing vtk [71/71] -- fbuild -dist clang -- 77x19
-> Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Tests/CMakeLib/CMakeFiles
testUVProcessChainHelper.dir/testUVProcessChainHelper.cxx.o <REMOTE: 172.28.
9.172>
-> Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Tests/CMakeLib/PseudoMemc
eck/CMakeFiles/pseudo_purity.dir/ret0.cxx.o <REMOTE: 172.28.79.172>
11>Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMake
iles/cmake.dir/cmCursesCacheEntryComposite.cxx.o <LOCAL>
10>Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMake
iles/cmake.dir/cmCursesPathWidget.cxx.o <LOCAL>
13> Exe: /Users/hoffman/Work/My Builds/cmake-fbuild/bin/cpack
4> Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/CMakeFiles/cmake.d
r/cmcmd.cxx.o <LOCAL>
14>Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMake
iles/cmake.dir/cmCursesMainForm.cxx.o <LOCAL>
16>Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/CMakeFiles/cmake.d
r/cmmain.cxx.o <LOCAL>
13>Obj: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/CMakeFiles/CTestLi
.dir/cmCTest.cxx.o <LOCAL>
99.8 % [*****] 31s (8/8)+(9/9) |
```

FBUILDWorker v1.16 | 'traffamadore' | 1 Connections

Current Mode: Work For Others Always Threshold: 20% Using: 8 CPUs (50.0%)

CPU	Host	Status
1	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMakeFiles/c
2	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMakeFiles/c
3	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Tests/CMakeLib/PseudoMemcheck/C
4	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMakeFiles/c
5	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Source/CursesDialog/CMakeFiles/c
6	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Tests/CMakeLib/CMakeFiles/testUV
7	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Tests/CMakeLib/PseudoMemcheck/C
8	traffamadore	/Users/hoffman/Work/My Builds/cmake-fbuild/Tests/CMakeLib/PseudoMemcheck/C
9		(Disabled)
10		(Disabled)

Terminal output continues with compiler flags and file paths.

Bill Hoffman's local experiment

FASTBuild: reuse the work

Cache location configured

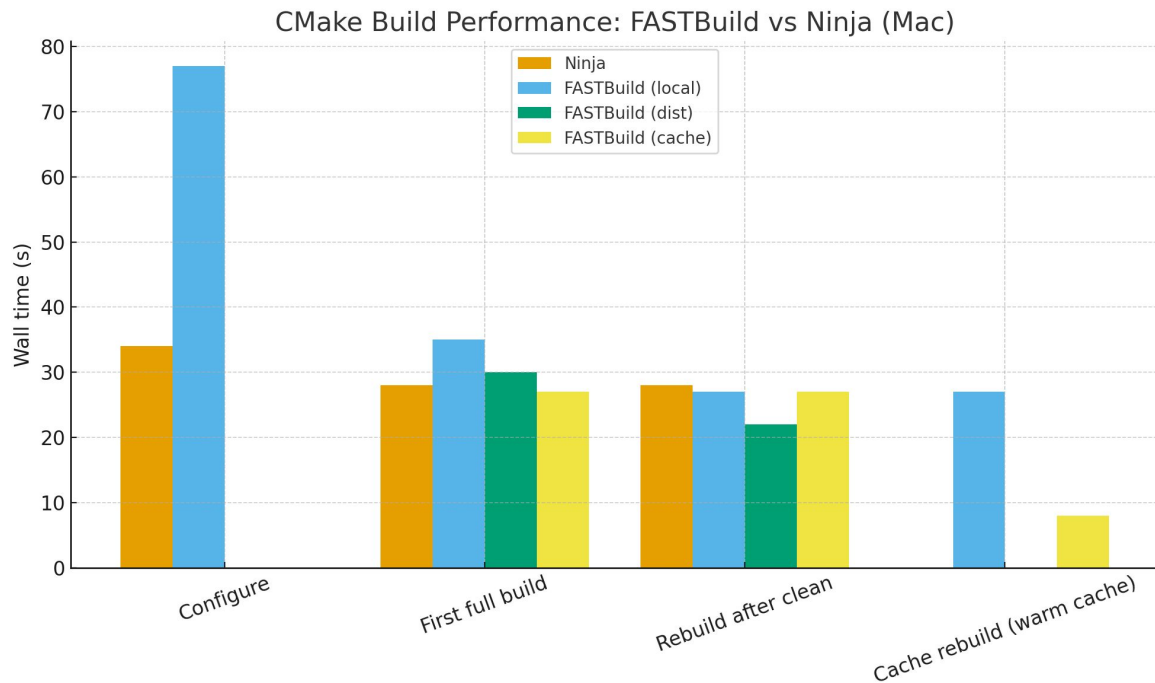
```
cmake --build build -- -cache
```

Same inputs. Cached results.

Clean the outputs.
Rebuild from a warm cache.

```
cmake-fbuild — Spack: Installing vtk [71/71] — -zsh • Emacs-arm64-11 — 77x19
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/ProcessUNIX.c.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/RegularExpression.cxx.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/Status.cxx.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/String.c.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/System.c.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/SystemInformation.cxx.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsys.dir/SystemTools.cxx.o
6> Remove: /Users/hoffman/Work/My Builds/cmake-fbuild/Source/kwsys/CMakeFiles
/cmsysTestDynload.dir/testDynload.c.o
FBuild: OK: clean
Time: 0.116s
hoffman@tralfamadore cmake-fbuild % fbuild -cache
```

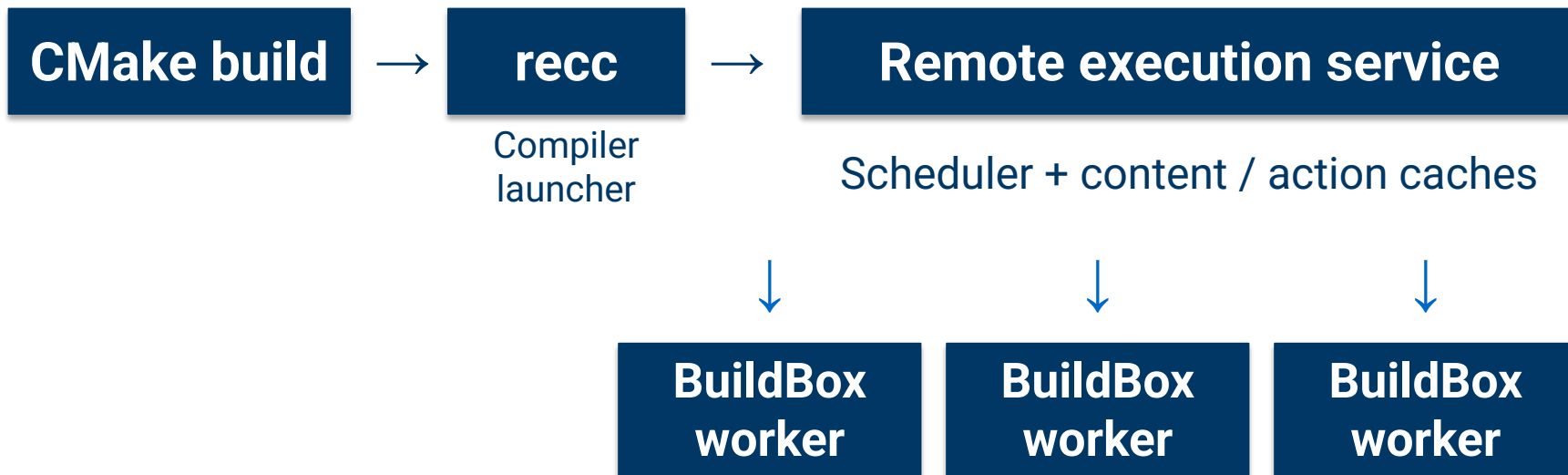
FASTBuild: measure the trade-offs



More configure time. The warm cache wins here.

CMake build on Mac / Bill Hoffman's measurements; workload-specific results

Assemble an open-source remote build service



Integrate the toolchains, storage, network and workers.

BuildGrid is one remote execution service; BuildBox supplies workers and recc supplies the compiler client.

So I tried it locally...

Windows laptop



macOS M1 laptop

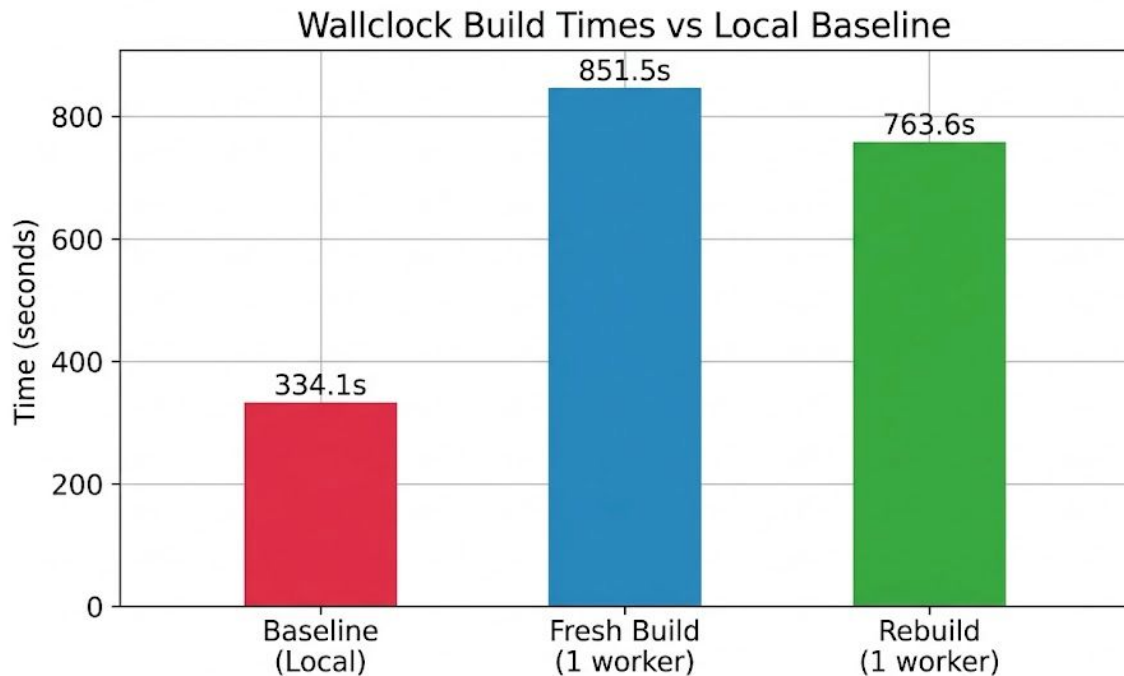
BuildGrid scheduler
PostgreSQL + cache
recc client

BuildBox worker
One remote worker

Full CMake build / make -j20 / Docker Compose

Could this small setup accelerate the build?

It worked. It was slower.



Local prototype / 334.1 s local; 851.5 s fresh remote build; 763.6 s remote rebuild / one worker

A working prototype is only the beginning

My small setup

One remote worker

Shared laptop resources

Service and network overhead

A production deployment

Dedicated worker capacity

Engineered storage and network

Repeatable toolchains

We work with a customer using this at massive scale.

The infrastructure takes engineering.

CMake RE: a commercial route to scale

tipi.build by EngFlow

CMake project → cmake-re → EngFlow build fleet

Prepare

Mirror inputs
Containerized tasks

Execute

Remote workers
Shared caching

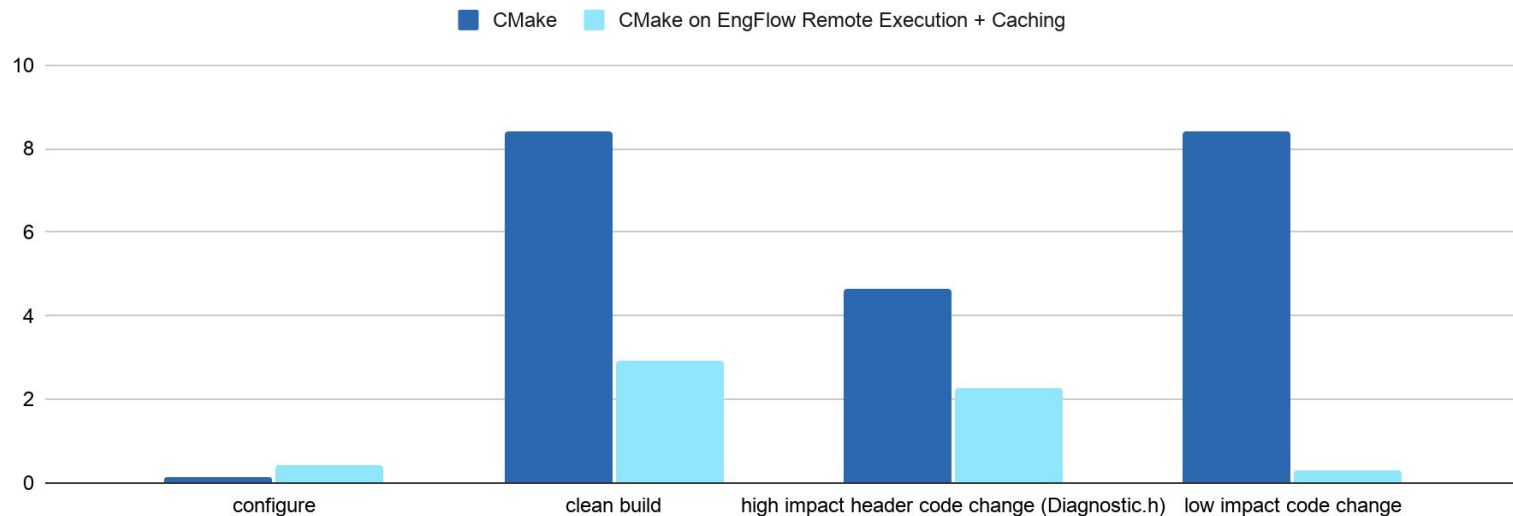
Adopt

Integration help
and support

An option for teams that want help putting the stack into service.

CMake RE: LLVM / Clang results

LLVM / Clang build average duration (min)



Different changes. Different caching opportunities.

[EngFlow-provided results / different workload and hardware from the laptop experiments](#)

Could CMake make this easier still?

CMake's model → BuildBox integration

Could a generator simplify the integration?

A possible future direction

We started with one compiler command

The starting point

```
g++ hello.cpp -o hello
```



Targets and their relationships



Build across platforms
Share package metadata
Describe what ships

Test continuously
Schedule modules
Measure and scale builds

CMake captures the expertise behind the work.

Your build works. What could it do better?



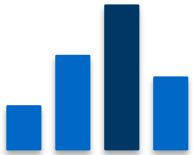
Model requirements with targets

Give downstream users the knowledge they need.



Publish CPS. Try modules.

Put the ecosystem work to use.



Measure your builds.

Find the bottlenecks. Keep the whole team moving.

26 years of building CMake together

To the users

who bring real problems, report bugs, and help each other.

Contributors and maintainers

turning hard problems
into shared solutions.



Kitware's customers

whose projects and
support
help CMake grow.

Thank you, CMake community.

CMake: Making Hard Things Easier



Capture the expertise once.

Make it repeatable.

Keep it working everywhere.



Bill Hoffman

Questions?

The slides will be available...



The slides will be available... as vibe-coded Python.

```
build_initial_deck.py / selected excerpts
for stage in range(5):
    compiler_slide(prs, stage)
cps_sbom_slides(prs)
sbom_example_slides(prs)
modules_slides(prs)
instrumentation_slides(prs)
scaling_slides(prs)
prs.save(OUTPUT)
```

Some assembly required.